

Python > langchain-community > document_loaders > pdf > PyPDFLoader

Class

 v0.4.2 (latest) ▼

• Since v0.3

PyPDFLoader

Load and parse a PDF file using 'pypdf' library.

This class provides methods to load and parse PDF documents, supporting various configurations such as handling password-protected files, extracting images, and defining extraction mode. It integrates the `pypdf` library for PDF processing and offers both synchronous and asynchronous document loading.

Examples: Setup:

```
.. code-block:: bash
```

```
pip install -U langchain-community pypdf
```

Instantiate the loader:

```
.. code-block:: python
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader(  
    file_path = "./example_data/layout-parser-paper.pdf",  
    # headers = None  
    # password = None,  
    mode = "single",  
    pages_delimiter = "
```

```
", # extract_images = True, # images_parser = RapidOCRBlobParser(), )
```

Lazy load documents:

```
.. code-block:: python
```

```
docs = []  
docs_lazy = loader.lazy_load()  
  
for doc in docs_lazy:  
    docs.append(doc)  
print(docs[0].page_content[:100])
```

```
print(docs[0].metadata)
```

Load documents asynchronously:

.. code-block:: python

```
docs = await loader.aload()
print(docs[0].page_content[:100])
print(docs[0].metadata)
```

```
PyPDFLoader(
    self,
    file_path: Union[str, PurePath],
    password: Optional[Union[str, bytes]] = None,
    headers: Optional[dict] = None,
    extract_images: bool = False,
    *,
    mode: Literal['single', 'page'] = 'page',
    images_parser: Optional[BaseImageBlobParser] = None,
    images_inner_format: Literal['text', 'markdown-img', 'html-img'] = 'text',
    pages_delimiter: str = _DEFAULT_PAGES_DELIMITER,
    extraction_mode: Literal['plain', 'layout'] = 'plain',
    extraction_kwargs: Optional[dict] = None
)
```

BASES

[BasePDFLoader](#)

USED IN DOCS

[Azure blob storage loader integration](#)

[Azure Cosmos DB NoSQL integration](#)

[Google cloud storage file integration](#)

[Google Vertex AI vector search integration](#)

Parameters

Name	Type	Description
<code>file_path</code> *	<code>Union[str, PurePath]</code>	The path to the PDF file to be loaded.
<code>headers</code>	<code>Optional[dict]</code>	Default: <code>None</code> Optional headers to use for GET request to download a file from a web path.
<code>password</code>	<code>Optional[Union[str, bytes]]</code>	Default: <code>None</code> Optional password for opening encrypted PDFs.
<code>mode</code>	<code>Literal['single', 'page']</code>	Default: <code>'page'</code> The extraction mode, either "single" for the entire document or "page" for page-wise extraction.
<code>pages_delimiter</code>	<code>str</code>	Default: <code>_DEFAULT_PAGES_DELIMITER</code> A string delimiter to separate pages in single-mode extraction.
<code>extract_images</code>	<code>bool</code>	Default: <code>False</code> Whether to extract images from the PDF.
<code>images_parser</code>	<code>Optional[BaseImageBlobParser]</code>	Default: <code>None</code> Optional image blob parser.
<code>images_inner_format</code>	<code>Literal['text', 'markdown-img', 'html-img']</code>	Default: <code>'text'</code> The format for the parsed output. "text" = return the content as is "markdown-img" = wrap the content into an image markdown link, w/ link pointing to (<code>![body](#)</code>) "html-img" = wrap the content as the <code>alt</code> text of an tag and link to (<code></code>)

Name	Type	Description
extraction_mode	Literal['plain', 'layout']	Default: 'plain'
		"plain" for legacy functionality, "layout" extract text in a fixed width format that closely adheres to the rendered layout in the source pdf
extraction_kwargs	Optional[dict]	Default: None
		Optional additional parameters for the extraction process.

Constructors

constructor	__init__	
	Name	Type
	file_path	Union[str, PurePath]
	password	Optional[Union[str, bytes]]
	headers	Optional[dict]
	extract_images	bool
	mode	Literal['single', 'page']
	images_parser	Optional[BaseImageBlobParser]
	images_inner_format	Literal['text', 'markdown-img', 'html-img']
	pages_delimiter	str
	extraction_mode	Literal['plain', 'layout']
	extraction_kwargs	Optional[dict]

Attributes

attribute **parser**



Methods

method **lazy_load**



Lazy load given path as pages. Insert image, if possible, between two paragraphs. In this way, a paragraph can be continued on the next page.

Inherited from BasePDFLoader

ATTRIBUTES

- A **file_path**
- A **web_path** : None
- A **headers** : headers
- A **temp_dir**
- A **source** : str

Inherited from BaseLoader (langchain_core)

METHODS

- M **load**
- M **aload**
- M **load_and_split**
- M **alazy_load**

 [View source on GitHub](#)